UNIVERSIDADE FEDERAL DO CEARÁ

Campus Quixadá

QXD0068 - Reuso de Software

Prof. Francisco Victor da Silva Pinheiro

TRABALHO 1 - REFATORAÇÃO COM PADRÕES DE PROJETO EM SISTEMA LEGADO

Estudo de caso: Spring PetClinic

Debora Silva Viana - 557337

Anaildo do Nascimento Silva - 552836

Quixadá–CE
2026

Sumário

1. Introdução

2. Caracterização e Diagnóstico do Sistema Legado

3. Padrões de Projeto Selecionados

4. Refatorações Realizadas

5. Análise dos Resultados

6. Conclusão

7. Referências

Repositório: https://github.com/deboravians/spring-petclinic

Original: v1.0-original | Refatorada: v2.0-refatorada

1. Introdução

A reutilização de software depende não apenas da disponibilidade de código, mas de sua organização para que comportamentos possam ser compreendidos, reaproveitados e modificados com menor risco. A refatoração altera a estrutura interna do programa procurando preservar seu comportamento observável (FOWLER, 2018). Os padrões de projeto GoF oferecem soluções recorrentes, descritas por responsabilidades e colaborações entre objetos (GAMMA et al., 1994).

Neste trabalho, foi selecionado o Spring PetClinic, aplicação de demonstração do ecossistema Spring para o gerenciamento de uma clínica veterinária. A escolha considerou a disponibilidade de código aberto, os testes automatizados e a possibilidade de estudar o funcionamento do sistema em um prazo curto. Por ser uma aplicação relativamente pequena e organizada, as refatorações foram limitadas a problemas concretos encontrados no código, evitando introduzir abstrações sem justificativa.

O objetivo é identificar dificuldades de manutenção e reutilização e aplicar quatro padrões GoF - Strategy, Template Method, Facade e Builder - comparando a estrutura original (tag v1.0-original) com a versão modificada (tag v2.0-refatorada). O fork está disponível em https://github.com/deboravians/spring-petclinic.

2. Caracterização e Diagnóstico do Sistema Legado

2.1 Caracterização do sistema

O PetClinic permite pesquisar, cadastrar e editar proprietários; cadastrar e modificar animais de estimação; registrar visitas; e consultar veterinários e especialidades. A aplicação utiliza Java, Spring Boot, Spring MVC, Thymeleaf e Spring Data JPA. O perfil padrão emprega H2 em memória; há suporte alternativo a MySQL e PostgreSQL. Os pacotes principais são model, owner, vet e system. O esforço concentrou-se no pacote owner, responsável por proprietários, pets, visitas, controladores e validação.

2.2 Problemas identificados

Analisando o código da versão original (tag v1.0-original), identificamos quatro problemas concretos, todos no pacote owner:

Validação espalhada e duplicada. As regras de validação de pets ficavam em três lugares: no PetValidator (nome obrigatório e com até 30 caracteres, tipo e data de nascimento obrigatórios) e nos métodos processCreationForm e processUpdateForm do PetController (nome duplicado e data de nascimento no futuro). O bloco que rejeita data de nascimento no futuro estava copiado de forma idêntica nos dois métodos. Além disso, a checagem de nome duplicado é diferente em cada operação: na criação, basta o dono já ter um pet salvo com o mesmo nome; na edição, o pet pode manter o próprio nome, então o código precisa comparar os identificadores.

Fluxo de formulário duplicado. Os dois métodos POST do PetController seguiam a mesma sequência: validar, voltar ao formulário em caso de erro, gravar dentro de um try, capturar DataIntegrityViolationException, verificar se a violação era de nome duplicado, registrar a mensagem de sucesso e redirecionar. Até o bloco catch era igual nos dois métodos; mudavam apenas a regra de duplicidade, a forma de gravar e o texto da mensagem.

Acesso direto e repetido aos repositórios. O trecho findById(...).orElseThrow(...), usado para buscar o proprietário, aparecia cinco vezes em três controllers (OwnerController, PetController e VisitController), e nem sempre com a mesma mensagem de erro. O tratamento de pet inexistente também variava: o VisitController lançava exceção, enquanto o PetController.findPet retornava null.

Construção de objetos verbosa nos testes. As classes de teste criavam proprietários, pets e visitas com new seguido de longas sequências de setters; só o método george() do OwnerControllerTests tinha 15 linhas. Havia ainda uma dependência de ordem escondida: o setId(1) do pet precisava ser chamado depois de

owner.addPet(max), porque addPet ignora pets com identificador repetido. Nada no código deixava isso explícito.

Esses problemas não impediam o funcionamento do sistema, mas tornavam algumas alterações mais dispersas. Em particular, havia risco de alterar um fluxo e esquecer o equivalente. A refatoração procurou centralizar políticas reutilizáveis e tornar explícitos os pontos em que as operações precisam variar.

2.3 Oportunidades de reutilização

Os problemas acima também mostram onde havia comportamento que poderia ser escrito uma vez e reaproveitado. A regra de data de nascimento no futuro vale tanto para a criação quanto para a edição de pets. O roteiro de processamento do formulário (validar, gravar, tratar duplicidade e redirecionar) é o mesmo nas duas operações. A busca do proprietário pelo identificador é necessária em três controllers diferentes. E a montagem de proprietários com pets e visitas se repete em várias classes de teste. Foram esses pontos que orientaram a escolha dos padrões apresentados no próximo capítulo.

3. Padrões de Projeto Selecionados

Problema	Padrão GoF	Justificativa	Benefício estrutural
Regras de validação dispersas	Strategy (comportamental)	Encapsular regras e combiná-las por operação	Novas regras isoladas do controller
Algoritmo duplicado em formulários	Template Method (comportamental)	Fixar o roteiro e especializar etapas	Tratamento compartilhado de erros
Acesso repetido a repositórios	Facade (estrutural)	Oferecer uma entrada única ao subsistema	Menor dependência dos controllers em JPA
Construção repetitiva de fixtures	Builder (criacional)	Expressar dados e proteger ordem de montagem	Testes mais legíveis e consistentes

Strategy é representado por PetValidationRule, suas implementações e PetValidator. Template Method utiliza PetFormProcessor e as especializações CreatePetFormProcessor e UpdatePetFormProcessor. Facade utiliza ClinicFacade, consultada por controllers e processadores. Builder utiliza OwnerBuilder, PetBuilder e VisitBuilder, localizados em src/test. O Builder tem efeito nos testes, e não diretamente nas funcionalidades de produção.

4. Refatorações Realizadas

4.1 Strategy: regras de validação

Na versão original, o PetValidator tinha as regras escritas diretamente no método validate, e as regras que dependem do proprietário (nome duplicado e data no futuro) ficavam no PetController. O que justifica o Strategy aqui é que as regras variam conforme a operação: criação e edição precisam de checagens de nome duplicado diferentes, mas compartilham a regra de data. Com a refatoração, a interface PetValidationRule define o contrato validate(Pet, Errors), e cada regra virou uma classe que implementa esse contrato. O PetValidator passou a ser o contexto do padrão: recebe uma lista de regras pelo construtor e as executa sem saber quais são. O catálogo PetValidationRules monta os conjuntos de cada situação: fieldRules(), forCreation() e forUpdate(). Mantivemos as duas etapas de validação do sistema original (a de campos, feita pelo Spring durante o binding, e a específica de cada operação), porque juntá-las mudaria o momento em que cada erro aparece para o usuário.

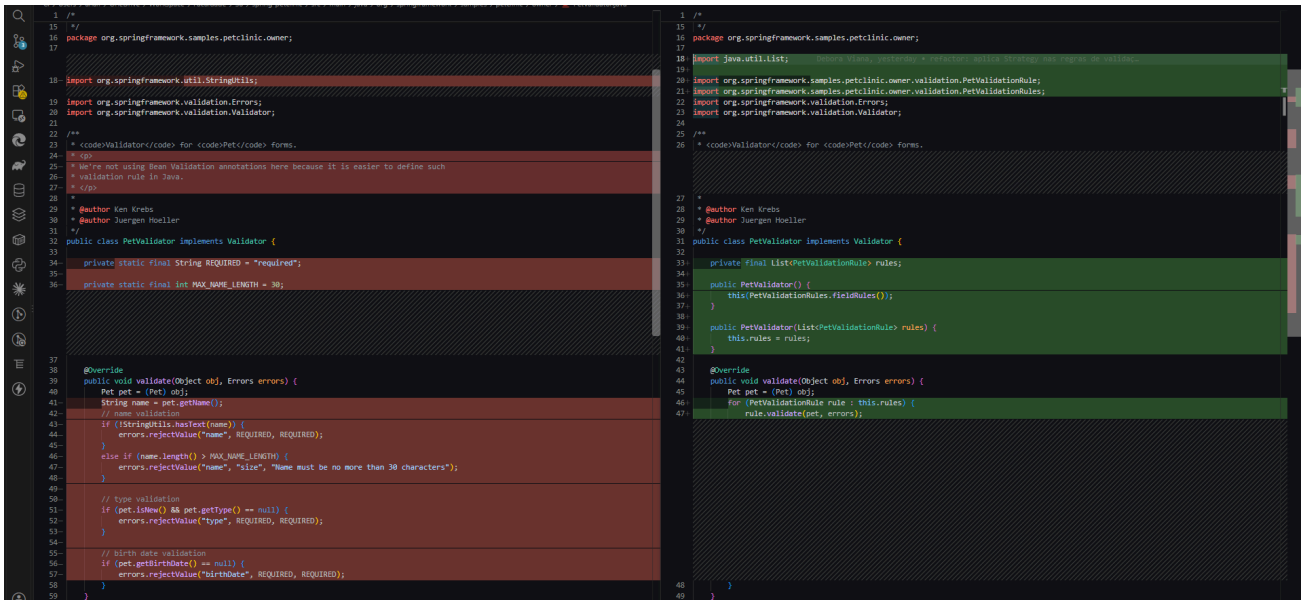


Figura 1 - PetValidator: regras incorporadas ao método (esquerda) e execução de estratégias (direita).

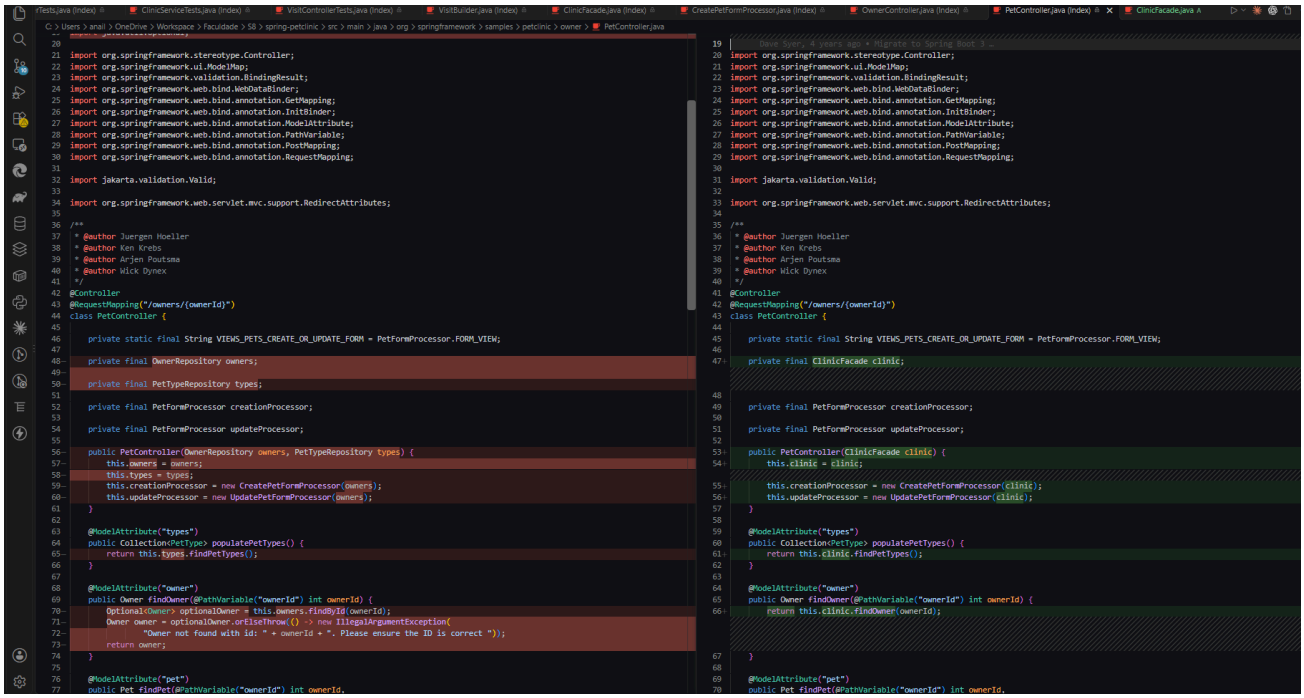
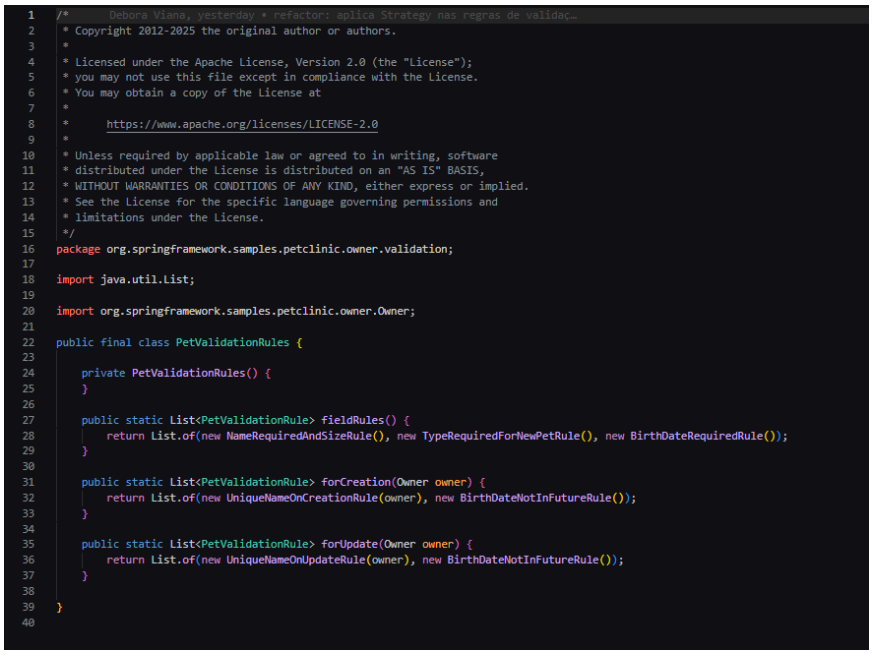


Figura 2 - Condicionais de validação retiradas do PetController na aplicação do Strategy.



```java
/*      Debora Viana, yesterday • refactor: aplica Strategy nas regras de validaç…
 * Copyright 2012-2025 the original author or authors.
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *      https://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.springframework.samples.petclinic.owner.validation;

import java.util.List;

import org.springframework.samples.petclinic.owner.Owner;

public final class PetValidationRules {

    private PetValidationRules() {
    }

    public static List<PetValidationRule> fieldRules() {
        return List.of(new NameRequiredAndSizeRule(), new TypeRequiredForNewPetRule(), new BirthDateRequiredRule());
    }

    public static List<PetValidationRule> forCreation(Owner owner) {
        return List.of(new UniqueNameOnCreationRule(owner), new BirthDateNotInFutureRule());
    }

    public static List<PetValidationRule> forUpdate(Owner owner) {
        return List.of(new UniqueNameOnUpdateRule(owner), new BirthDateNotInFutureRule());
    }

}

```

Figura 3 - Catálogo PetValidationRules: regras de campos, criação e edição.

A nova estrutura possibilita testar individualmente cenários como nascimento em data futura e duplicidade de nome. O custo é o aumento da quantidade de classes, aceitável neste estudo pelo ganho de separação e possibilidade de substituição.

Ganho para reuso e manutenção: a regra de data no futuro passou a existir uma única vez e é reaproveitada pelas duas operações. Para incluir uma regra nova, basta criar uma classe e adicioná-la ao catálogo, sem mexer no controller nem no PetValidator. Os casos das regras isoladas ficaram cobertos por testes próprios (PetValidationRulesTests).

4.2 Template Method: processamento de formulários

Antes da alteração, os dois métodos POST de PetController duplicavam verificações de erros, persistência protegida por try/catch e redirecionamentos. Depois, PetFormProcessor passou a oferecer process() como método final. As subclasses definem validationRules(), save() e successMessage(), mantendo o algoritmo compartilhado e a variação exigida em cada operação.

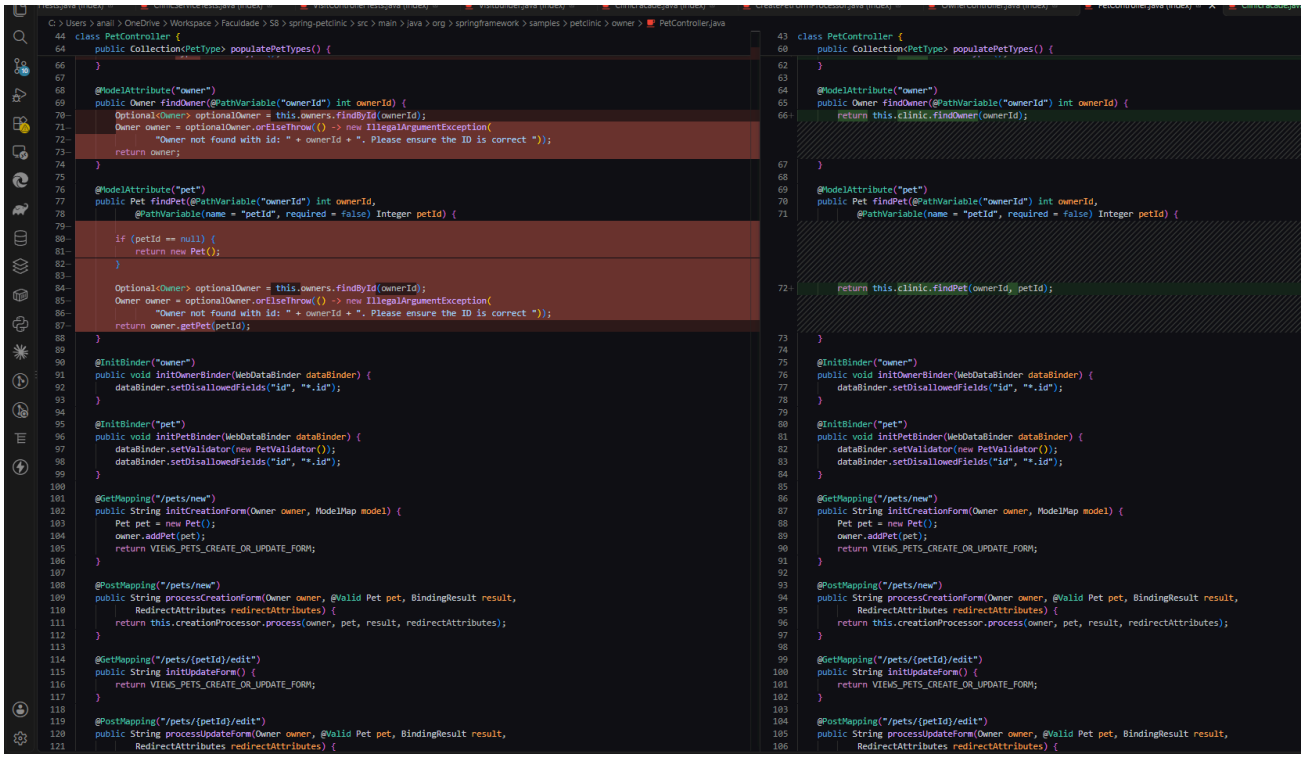
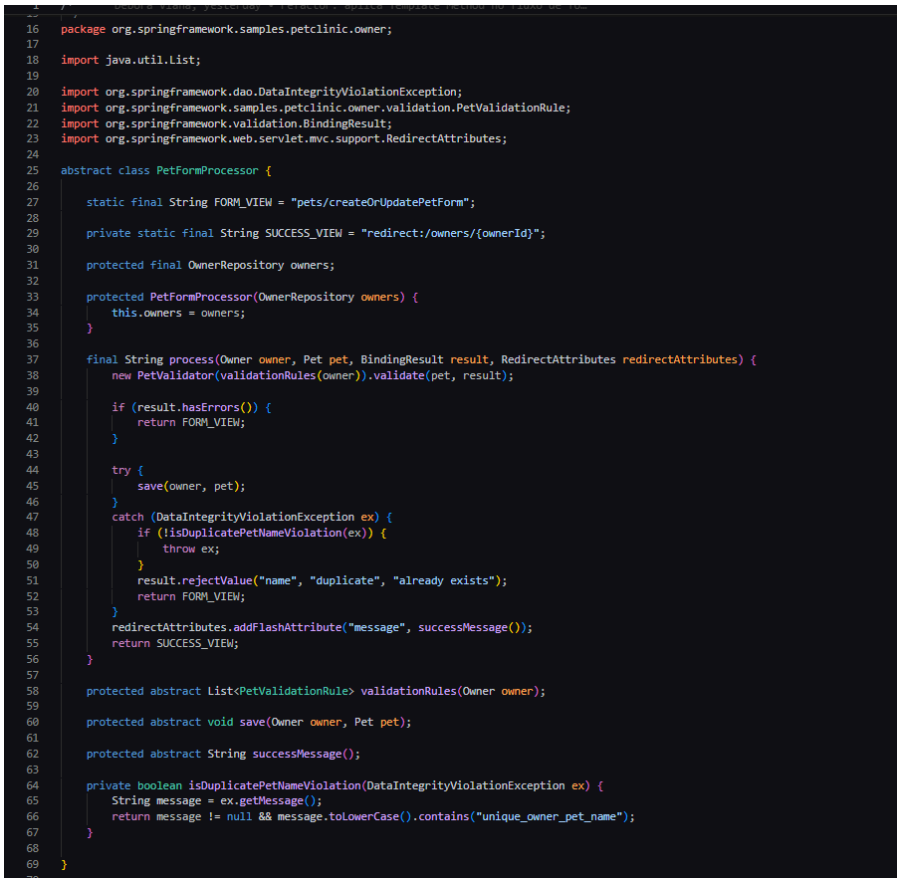


Figura 4 - PetController: substituição de dois fluxos repetidos por delegações aos processadores.



```java
package org.springframework.samples.petclinic.owner;

import java.util.List;

import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.samples.petclinic.owner.validation.PetValidationRule;
import org.springframework.validation.BindingResult;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

abstract class PetFormProcessor {

    static final String FORM_VIEW = "pets/createOrUpdatePetForm";

    private static final String SUCCESS_VIEW = "redirect:/owners/{ownerId}";

    protected final OwnerRepository owners;

    protected PetFormProcessor(OwnerRepository owners) {
        this.owners = owners;
    }

    final String process(Owner owner, Pet pet, BindingResult result, RedirectAttributes redirectAttributes) {
        new PetValidator(validationRules(owner)).validate(pet, result);

        if (result.hasErrors()) {
            return FORM_VIEW;
        }

        try {
            save(owner, pet);
        }
        catch (DataIntegrityViolationException ex) {
            if (!isDuplicatePetNameViolation(ex)) {
                throw ex;
            }
            result.rejectValue("name", "duplicate", "already exists");
            return FORM_VIEW;
        }
        redirectAttributes.addFlashAttribute("message", successMessage());
        return SUCCESS_VIEW;
    }

    protected abstract List<PetValidationRule> validationRules(Owner owner);

    protected abstract void save(Owner owner, Pet pet);

    protected abstract String successMessage();

    private boolean isDuplicatePetNameViolation(DataIntegrityViolationException ex) {
        String message = ex.getMessage();
        return message != null && message.toLowerCase().contains("unique_owner_pet_name");
    }

}
```

Figura 5 - Classe PetFormProcessor com o roteiro fixo e as operações abstratas.

```java
package org.springframework.samples.petclinic.owner;

import java.util.List;

import org.springframework.samples.petclinic.owner.validation.PetValidationRule;
import org.springframework.samples.petclinic.owner.validation.PetValidationRules;
import org.springframework.util.Assert;

class UpdatePetFormProcessor extends PetFormProcessor {

    UpdatePetFormProcessor(OwnerRepository owners) {
        super(owners);
    }

    @Override
    protected List<PetValidationRule> validationRules(Owner owner) {
        return PetValidationRules.forUpdate(owner);
    }

    /**
     * Updates the pet details if it exists or adds a new pet to the owner.
     * @param owner The owner of the pet
     * @param pet The pet with updated details
     */
    @Override
    protected void save(Owner owner, Pet pet) {
        Integer id = pet.getId();
        Assert.state(id != null, "'pet.getId()' must not be null");
        Pet existingPet = owner.getPet(id);
        if (existingPet != null) {
            // Update existing pet's properties
            existingPet.setName(pet.getName());
            existingPet.setBirthDate(pet.getBirthDate());
            existingPet.setType(pet.getType());
        }
        else {
            owner.addPet(pet);
        }
        this.owners.saveAndFlush(owner);
    }

    @Override
    protected String successMessage() {
        return "Pet details has been edited";
    }

}
```

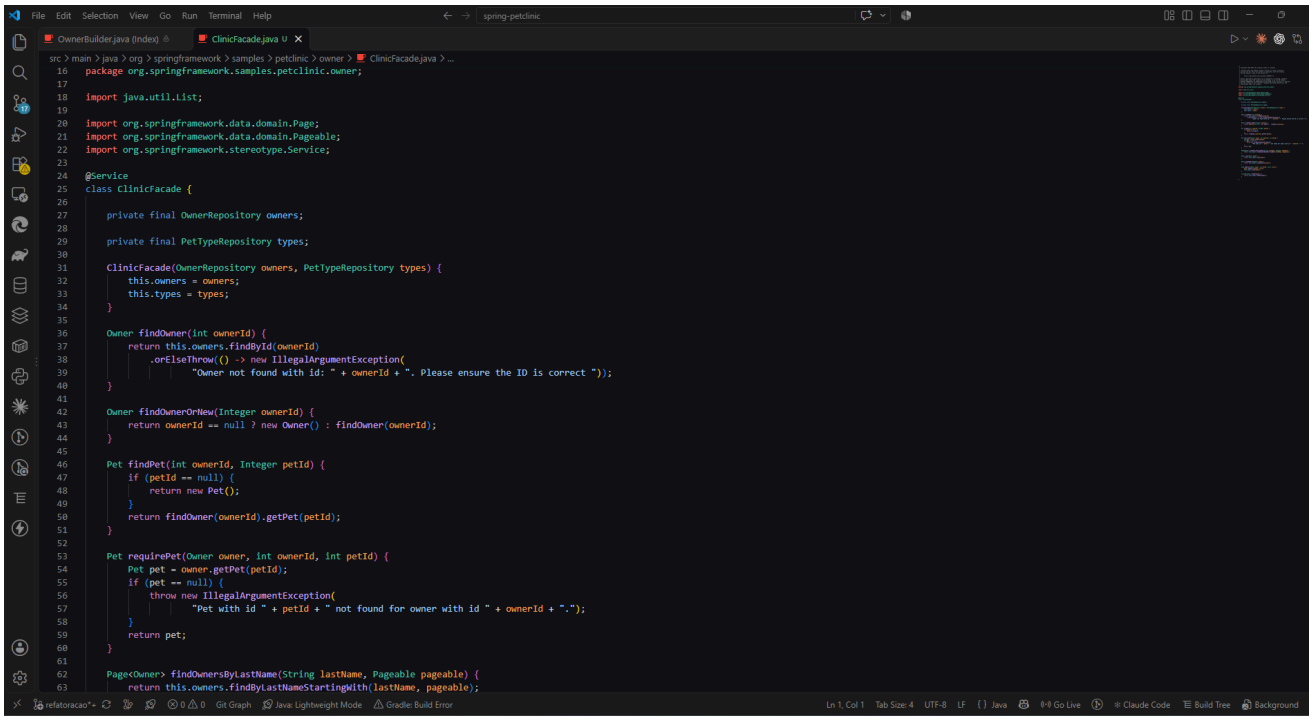
Figura 6 - Processador de atualização e suas operações específicas.

O tratamento de DataIntegrityViolationException e o redirecionamento ficam concentrados em process(), diminuindo o risco de divergência entre criação e atualização. O uso conjunto de Template Method e Strategy separa dois tipos de variação: os passos do fluxo e as regras usadas em cada passo.

Ganho para reuso e manutenção: o tratamento de DataIntegrityViolationException, que estava duplicado, agora existe só em PetFormProcessor, e cada método POST do controller ficou com uma linha. Como process() é final, uma nova operação sobre pets (por exemplo, transferir um pet para outro dono) só precisaria implementar os três passos variáveis, reaproveitando o fluxo inteiro sem risco de alterar a ordem das etapas.

4.3 Facade: acesso ao subsistema

Na versão anterior, OwnerController, PetController e VisitController acessavam diretamente os repositórios e reimplementavam buscas e tratamento de ausência de dados. ClinicFacade, marcada com @Service, agora concentra consultas de proprietários, obtenção de pets, consulta de tipos e gravações. Os controladores e processadores passam a utilizá-la como ponto de entrada. O método findPet preserva a criação de um objeto vazio quando o identificador do pet está ausente; requirePet mantém uma verificação explícita quando a operação exige um pet existente.



```java
package org.springframework.samples.petclinic.owner;

import java.util.List;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;

@Service
class ClinicFacade {

    private final OwnerRepository owners;

    private final PetTypeRepository types;

    ClinicFacade(OwnerRepository owners, PetTypeRepository types) {
        this.owners = owners;
        this.types = types;
    }

    Owner findOwner(int ownerId) {
        return this.owners.findById(ownerId)
            .orElseThrow(() -> new IllegalArgumentException(
                    "Owner not found with id: " + ownerId + ". Please ensure the ID is correct "));
    }

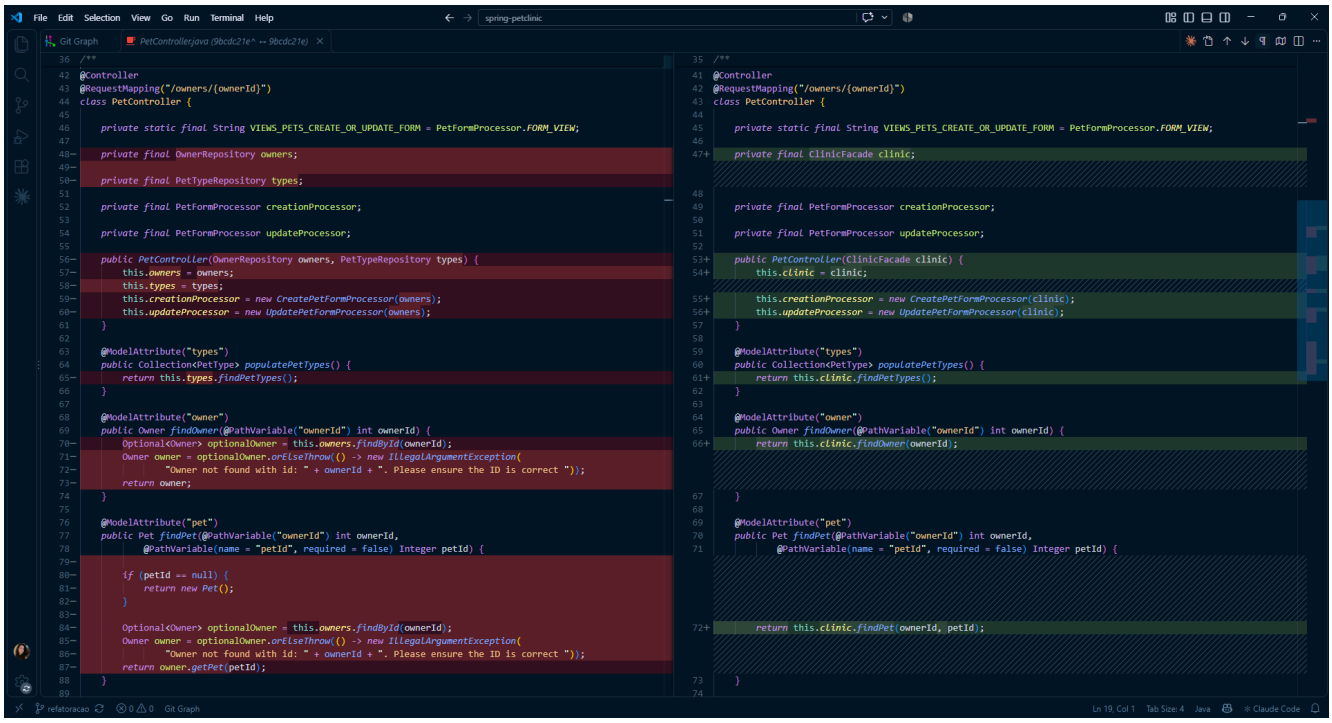
    Owner findOwnerOrNew(Integer ownerId) {
        return ownerId == null ? new Owner() : findOwner(ownerId);
    }

    Pet findPet(int ownerId, Integer petId) {
        if (petId == null) {
            return new Pet();
        }
        return findOwner(ownerId).getPet(petId);
    }

    Pet requirePet(Owner owner, int ownerId, int petId) {
        Pet pet = owner.getPet(petId);
        if (pet == null) {
            throw new IllegalArgumentException(
                    "Pet with id " + petId + " not found for owner with id " + ownerId + ".");
        }
        return pet;
    }

    Page<Owner> findOwnersByLastName(String lastName, Pageable pageable) {
        return this.owners.findByLastNameStartingWith(lastName, pageable);
```

Figura 7 - ClinicFacade: centralização de consultas, gravações e acesso ao subsistema.



```java
@Controller
@RequestMapping("/owners/{ownerId}")
class PetController {

    private static final String VIEWS_PETS_CREATE_OR_UPDATE_FORM = PetFormProcessor.FORM_VIEW;

    private final OwnerRepository owners;

    private final PetTypeRepository types;

    private final PetFormProcessor creationProcessor;

    private final PetFormProcessor updateProcessor;

    public PetController(OwnerRepository owners, PetTypeRepository types) {
        this.owners = owners;
        this.types = types;
        this.creationProcessor = new CreatePetFormProcessor(owners);
        this.updateProcessor = new UpdatePetFormProcessor(owners);
    }

    @ModelAttribute("types")
    public Collection<PetType> populatePetTypes() {
        return this.types.findPetTypes();
    }

    @ModelAttribute("owner")
    public Owner findOwner(@PathVariable("ownerId") int ownerId) {
        Optional<Owner> optionalOwner = this.owners.findById(ownerId);
        Owner owner = optionalOwner.orElseThrow(() -> new IllegalArgumentException(
                "Owner not found with id: " + ownerId + ". Please ensure the ID is correct "));
        return owner;
    }

    @ModelAttribute("pet")
    public Pet findPet(@PathVariable("ownerId") int ownerId,
            @PathVariable(name = "petId", required = false) Integer petId) {

        if (petId == null) {
            return new Pet();
        }

        Optional<Owner> optionalOwner = this.owners.findById(ownerId);
        Owner owner = optionalOwner.orElseThrow(() -> new IllegalArgumentException(
                "Owner not found with id: " + ownerId + ". Please ensure the ID is correct "));
        return owner.getPet(petId);
    }
```

```java
@Controller
@RequestMapping("/owners/{ownerId}")
class PetController {

    private static final String VIEWS_PETS_CREATE_OR_UPDATE_FORM = PetFormProcessor.FORM_VIEW;

    private final ClinicFacade clinic;

    private final PetFormProcessor creationProcessor;

    private final PetFormProcessor updateProcessor;

    public PetController(ClinicFacade clinic) {
        this.clinic = clinic;

        this.creationProcessor = new CreatePetFormProcessor(clinic);
        this.updateProcessor = new UpdatePetFormProcessor(clinic);
    }

    @ModelAttribute("types")
    public Collection<PetType> populatePetTypes() {
        return this.clinic.findPetTypes();
    }

    @ModelAttribute("owner")
    public Owner findOwner(@PathVariable("ownerId") int ownerId) {
        return this.clinic.findOwner(ownerId);

    }

    @ModelAttribute("pet")
    public Pet findPet(@PathVariable("ownerId") int ownerId,
            @PathVariable(name = "petId", required = false) Integer petId) {

        return this.clinic.findPet(ownerId, petId);

    }
```

Figura 8 - PetController: substituição das dependências de repositório pela ClinicFacade.

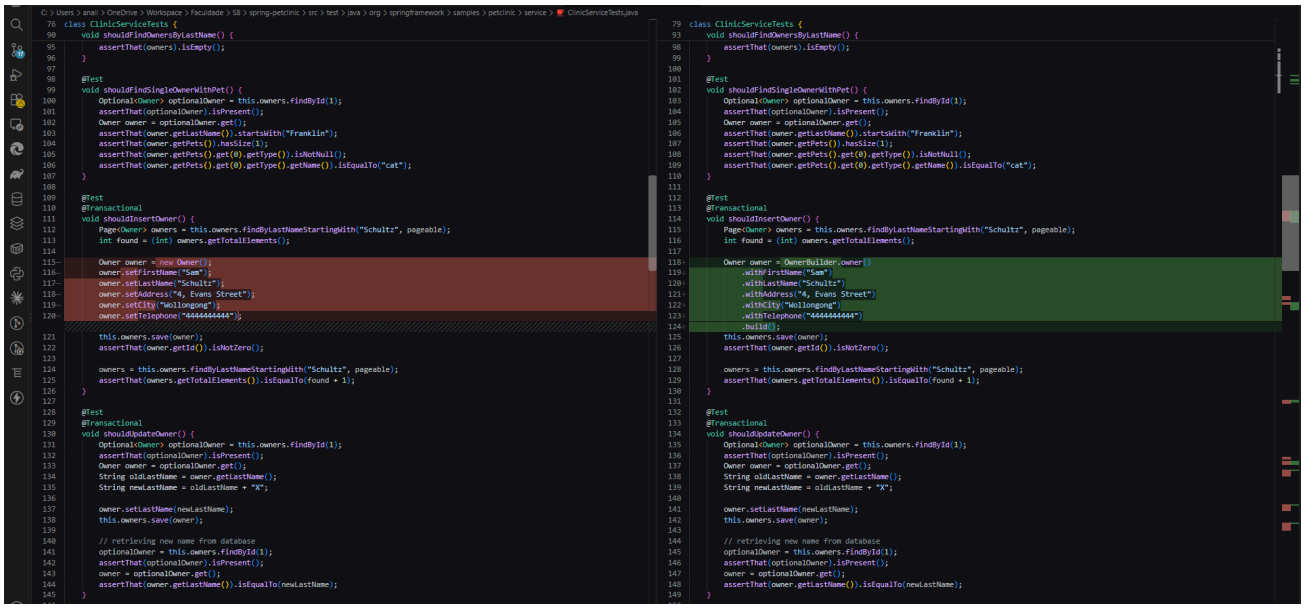


Figura 9 - OwnerController: chamadas diretas ao repositório substituídas pela fachada.

A mudança reduz o acoplamento dos controladores às APIs de persistência, mas adiciona um nível de indireção. Como alguns métodos da fachada apenas delegam, o benefício principal é a consistência de acesso e a possibilidade de centralizar mudanças futuras, não um ganho de desempenho demonstrado.

Ganho para reuso e manutenção: a busca do proprietário, antes repetida cinco vezes, passou a ser um único método reutilizado pelos três controllers, sempre com a mesma mensagem de erro. Uma mudança na forma de acessar os dados fica restrita à ClinicFacade. A diferença de comportamento entre findPet e requirePet foi mantida de propósito, já que corrigi-la alteraria o funcionamento do sistema; com a fachada, porém, ela ficou visível em um único lugar.

4.4 Builder: criação de objetos para testes

A montagem de fixtures por new e setters tornava os testes extensos e escondia uma dependência de ordem: Owner.addPet deve receber o pet antes de a atribuição de id potencialmente afetar sua inclusão na coleção. OwnerBuilder resolve o problema montando o pet sem id, adicionando-o ao proprietário e atribuindo o identificador em seguida; PetBuilder encapsula essas etapas. VisitBuilder auxilia na configuração seletiva de visitas.

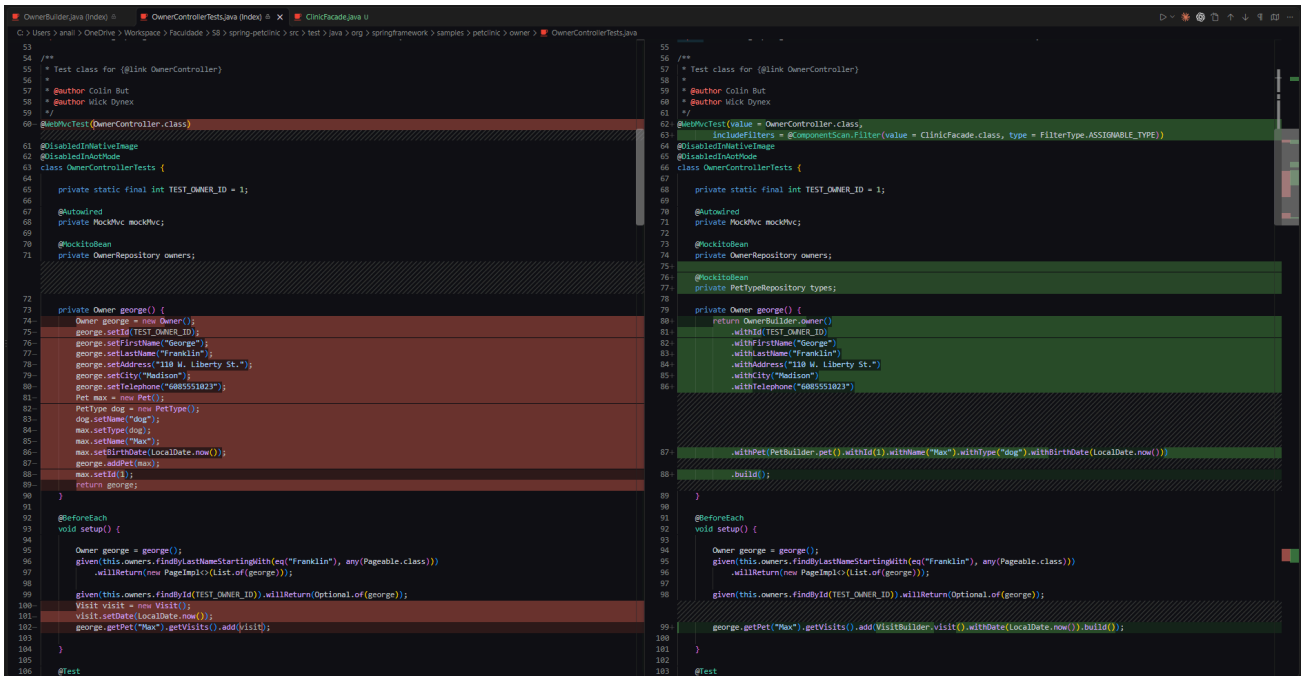


Figura 10 - OwnerControllerTests: fixture george() antes e depois do Builder.



Figura 11 - PetBuilder: construção gradual do pet e atribuição posterior de identificador.

No código publicado, OwnerBuilder.build() executa explicitamente petBuilder.buildWithoutId(), owner.addPet(pet) e petBuilder.assignId(pet). Essa é a evidência mais importante da solução: além da sintaxe encadeada, o padrão protege uma regra de construção. A refatoração tem alcance restrito aos testes automatizados.

Ganho para reuso e manutenção: os builders são reutilizados por quatro classes de teste (OwnerControllerTests, PetControllerTests, VisitControllerTests e ClinicServiceTests), e a regra de ordem entre addPet e o identificador fica encapsulada: quem escrever um teste novo não precisa conhecê-la. O ganho é restrito ao código de teste, mas torna mais simples e seguro criar novos cenários.

5. Análise dos Resultados

5.1 Validação funcional e automatizada

Para mostrar que o comportamento foi preservado, usamos duas evidências. A primeira é a suíte de testes automatizados do próprio projeto, complementada pelos testes que criamos para as regras de validação.

Depois dos quatro commits, executamos ./mvnw test, que terminou com BUILD SUCCESS: 84 testes executados, nenhuma falha, nenhum erro e dois ignorados (os testes de integração com MySQL e PostgreSQL, que dependem de Docker). Os testes de PetControllerTests, OwnerControllerTests e VisitControllerTests cobrem justamente os fluxos alterados, incluindo os casos de nome duplicado e de violação de unicidade vinda do banco. A segunda evidência é a execução manual da aplicação, registrada nas figuras a seguir, com consulta de proprietário, pet cadastrado, visitas registradas e listagem de veterinários.

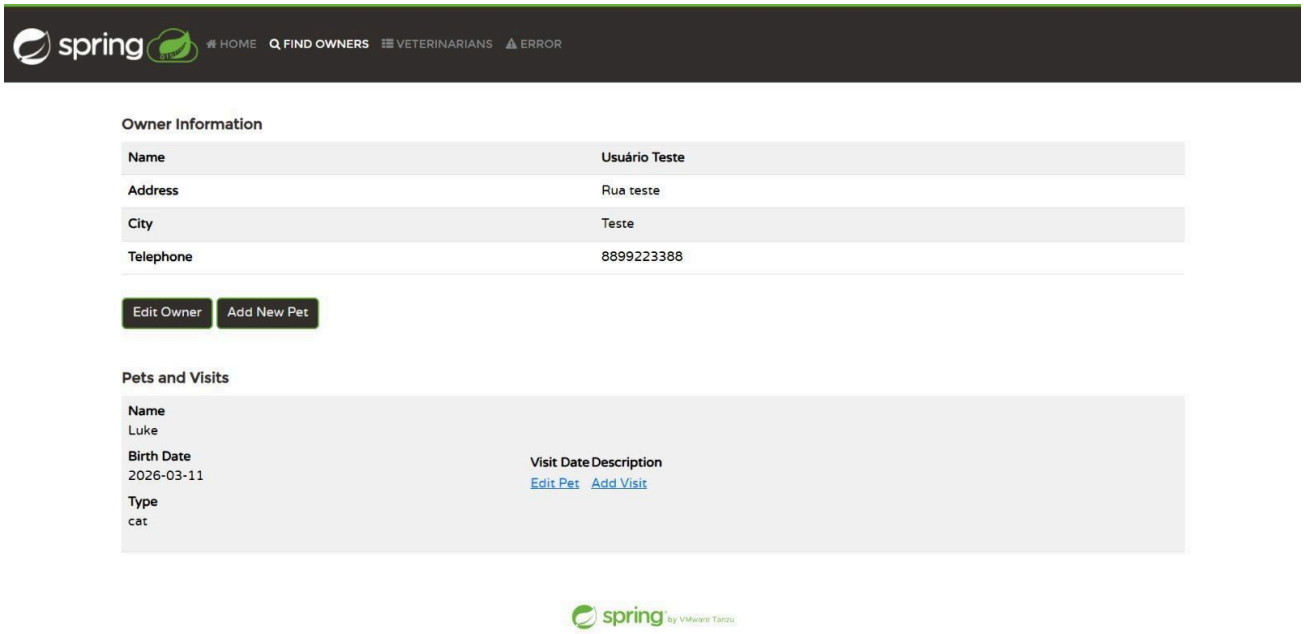


Figura 12 - Consulta de proprietário e visualização do animal cadastrado.

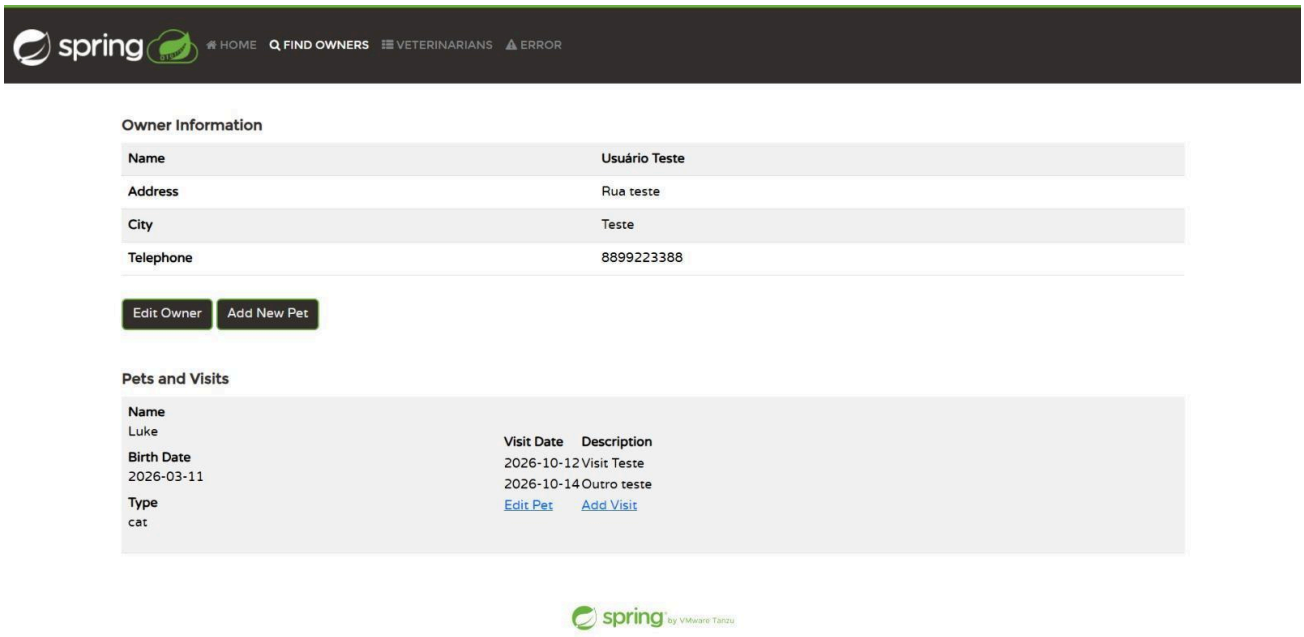


Figura 13 - Proprietário com visitas registradas para o pet.

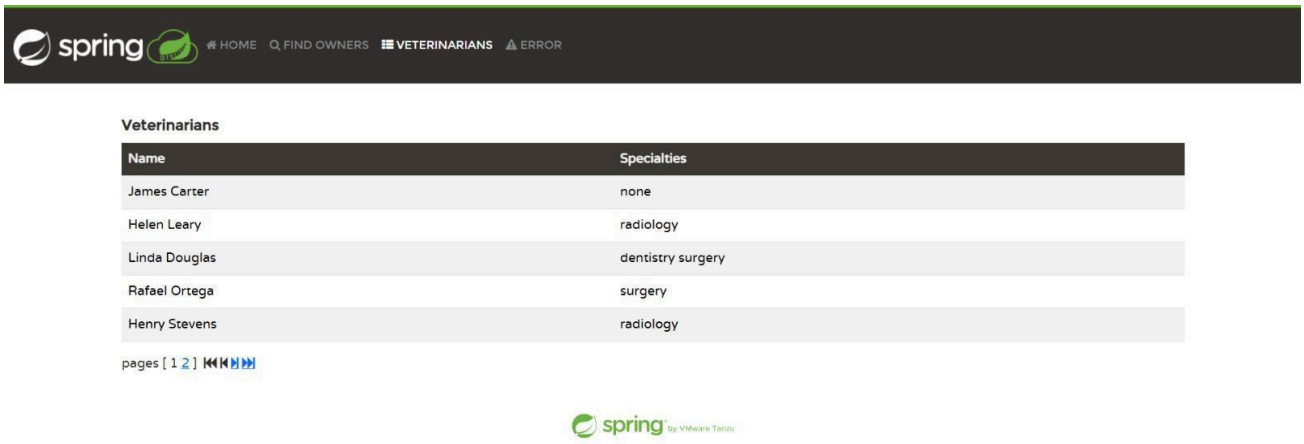


Figura 14 - Listagem paginada de veterinários.

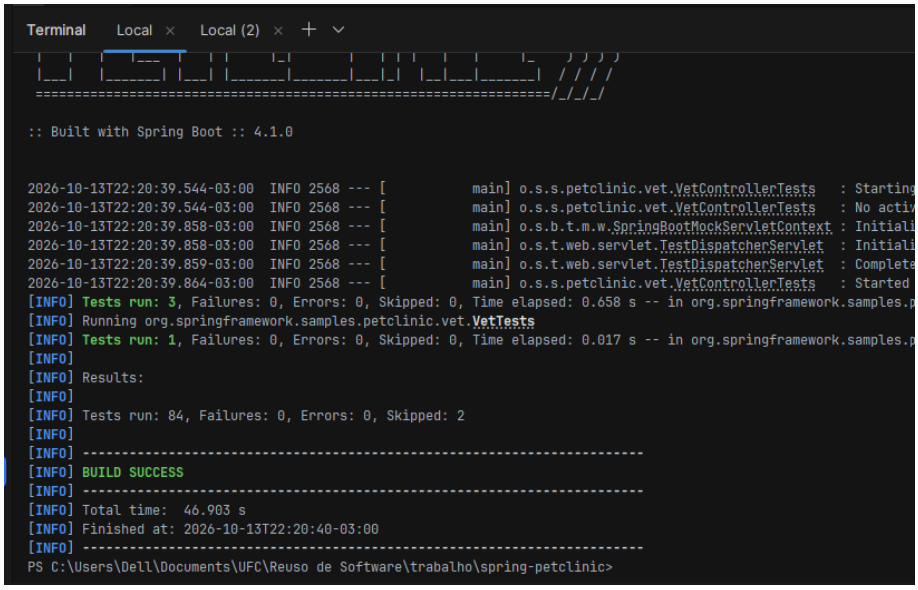


Figura 15 - Execução da suíte de testes após as refatorações.

5.2 Comparação da estrutura e potencial de reuso

No Strategy, a verificação de regras passa a depender de uma interface, enquanto a composição das regras fica no catálogo. No Template Method, o tratamento de erro e o redirecionamento passam a ter um ponto comum. Na Facade, os controllers deixam de depender diretamente de OwnerRepository e PetTypeRepository. No Builder, a montagem de fixtures e a atribuição de identificadores são organizadas em componentes próprios. Esses ganhos são qualitativos e verificáveis por inspeção dos trechos antes/depois; não foram calculadas métricas formais de acoplamento ou coesão.

Há custos de projeto: mais classes, abstrações e indireções. Em um sistema pequeno, parte das soluções poderia ser implementada com métodos auxiliares. A escolha dos quatro padrões se justifica neste estudo

porque cada um aborda uma dificuldade observada, e não porque a adoção de padrões seja sempre a opção mais simples.

5.3 Evidência do histórico de versões

A consulta à comparação GitHub v1.0-original...refatoracao identificou quatro commits: Strategy (a881fdf455), Template Method (7fa98e51be), Builder (a2083d5313) e Facade (9bcdc21ea8). As alterações publicadas abrangem 24 arquivos, entre adicionados e modificados. Essa informação documenta a evolução do código, mas não substitui o resultado dos testes da revisão final.

6. Conclusão

O estudo aplicou Strategy, Template Method, Facade e Builder em pontos delimitados do Spring PetClinic. A validação de pets foi organizada em regras substituíveis; os fluxos de criação e edição passaram a reutilizar um algoritmo comum; consultas e gravações ganharam um ponto de acesso central; e a criação de entidades nos testes passou a encapsular uma ordem de construção relevante. Os trechos de código e o histórico de commits sustentam a conclusão de que houve reorganização estrutural e maior localização das mudanças.

Os resultados devem ser interpretados no contexto de um projeto pequeno, relativamente limpo e com padrões fornecidos pelo framework. As refatorações não demonstram, por si só, ganhos de desempenho ou reduções quantitativas de acoplamento. A principal aprendizagem da dupla foi vincular cada padrão a um problema concreto e reconhecer o custo das abstrações introduzidas. A preservação do comportamento foi confirmada pela suíte de testes executada após todas as modificações. Como dificuldade, destacamos o prazo curto e o fato de o PetClinic já ser um projeto bem organizado, o que exigiu cuidado para não aplicar padrões onde eles não eram necessários.

7. Referências

FOWLER, Martin. Refactoring: Improving the Design of Existing Code. 2. ed. Boston: Addison-Wesley, 2018.

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Design Patterns: Elements of Reusable Object-Oriented Software. Reading: Addison-Wesley, 1994.

SPRING. Spring PetClinic sample application. Disponível em: https://github.com/spring-projects/spring-petclinic. Acesso em: 13 out. 2026.

SPRING. Spring Framework Reference Documentation. Disponível em: https://docs.spring.io/spring-framework/reference/. Acesso em: 13 out. 2026.

CODETRIAGE. Projetos open source. Disponível em: https://www.codetriage.com/. Acesso em: 13 out. 2026.